

Electro GO Application CI/CD Deployment on Microsoft Azure Using Azure DevOps Classic Pipelines

Table of Contents

1. Introduction
2. Project Overview
3. Aim
4. Objectives
5. Technology Stack
6. Architecture Diagram
7. CI/CD Workflow
8. Azure Resources
9. Azure Repos
10. Build Pipeline
11. Release Pipeline
12. Terraform Deployment
13. Application Deployment
14. Validation
15. Conclusion

1. Project Overview

Electro GO is an EV Charging Station Management Application developed using ASP.NET Core and MySQL. The project demonstrates an end-to-end CI/CD implementation using Azure DevOps Classic Pipelines. Azure Repos is used as the source code repository, the Build Pipeline compiles and tests the application, and the Release Pipeline provisions Azure infrastructure using Terraform before deploying the application automatically.

2. Aim

To automate the deployment of the Electro GO web application using Azure DevOps Classic CI/CD pipelines by integrating Azure Repos, Terraform, and Microsoft Azure.

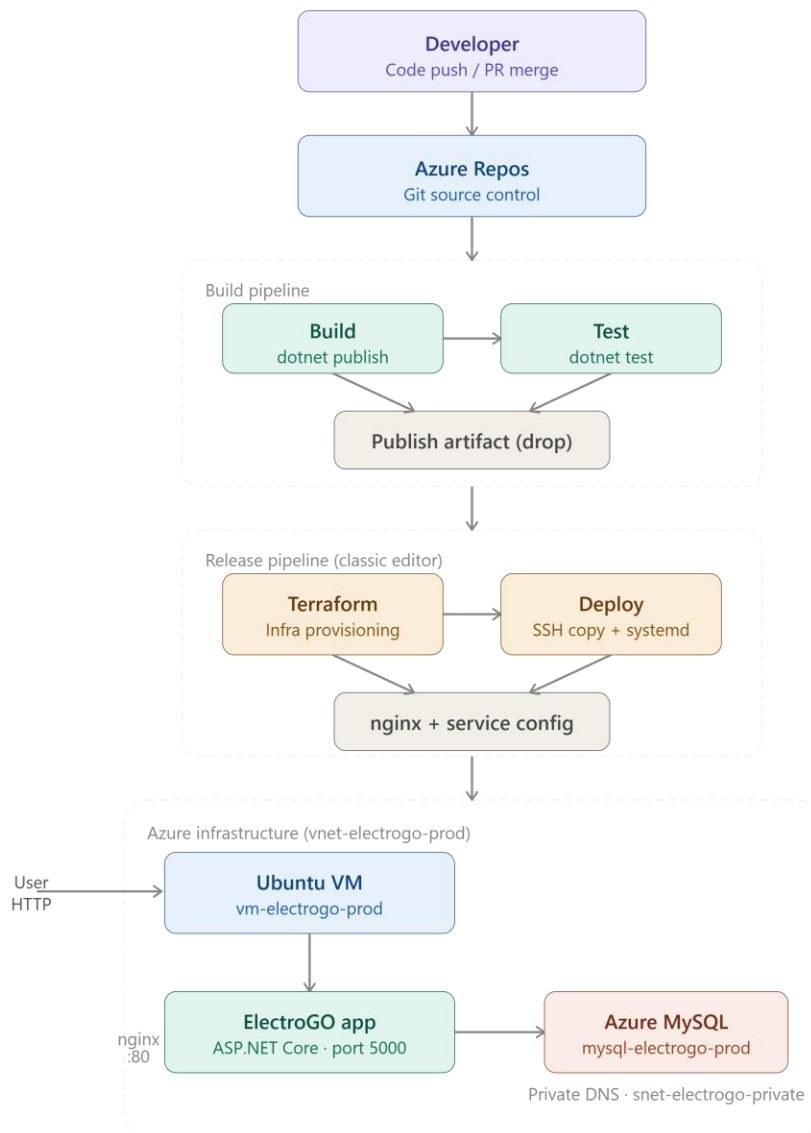
3. Objectives

- Create Azure infrastructure using Terraform
- Store source code in Azure Repos
- Build ASP.NET Core application
- Execute automated build validation
- Deploy Azure infrastructure
- Deploy application automatically
- Reduce manual deployment effort
- Implement Infrastructure as Code

4. Technology Stack

Component	Technology
Cloud	Microsoft Azure
CI/CD	Azure DevOps Classic Editor
Repository	Azure Repos
IaC	Terraform
Programming Language	ASp.NET
Framework	ASP.NET Core
Database	MySQL
Operating System	Ubuntu Linux
Web Server	Nginx

5. Architecture Diagram



6. Azure Repository

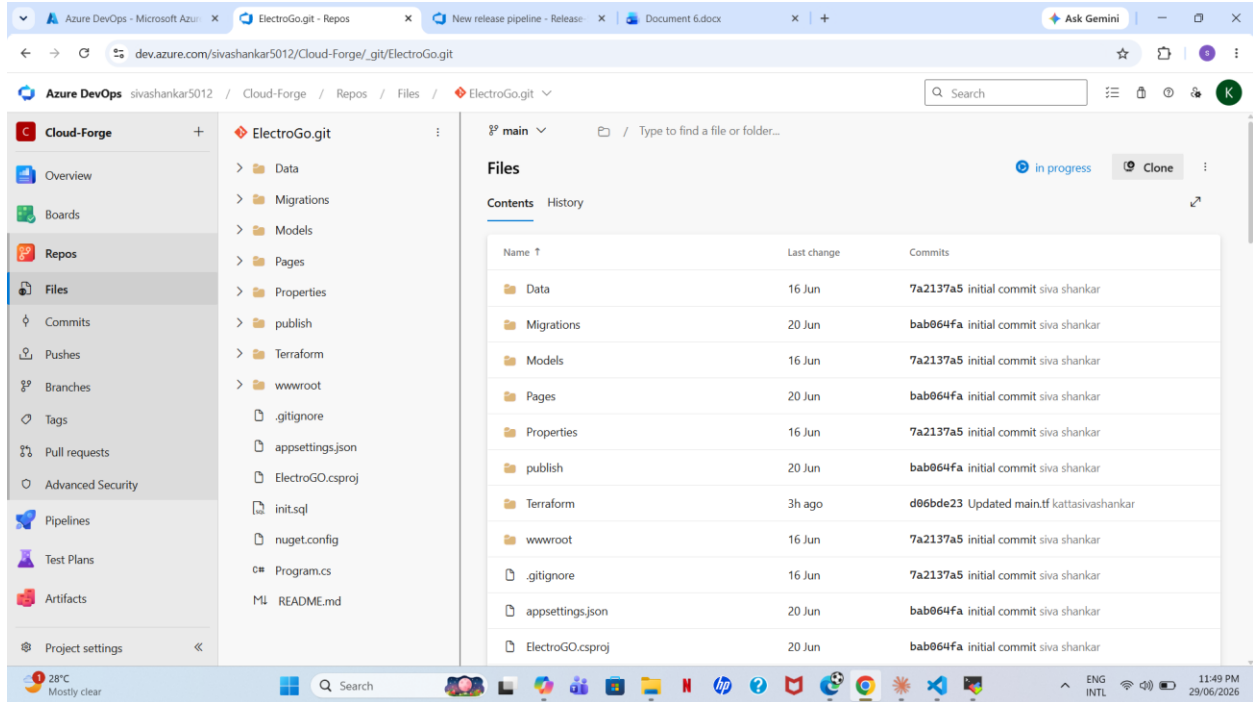
Purpose

Azure Repos stores the application source code and Terraform configuration. Whenever code changes are committed, the Build Pipeline can use the latest version of the project.

Procedure

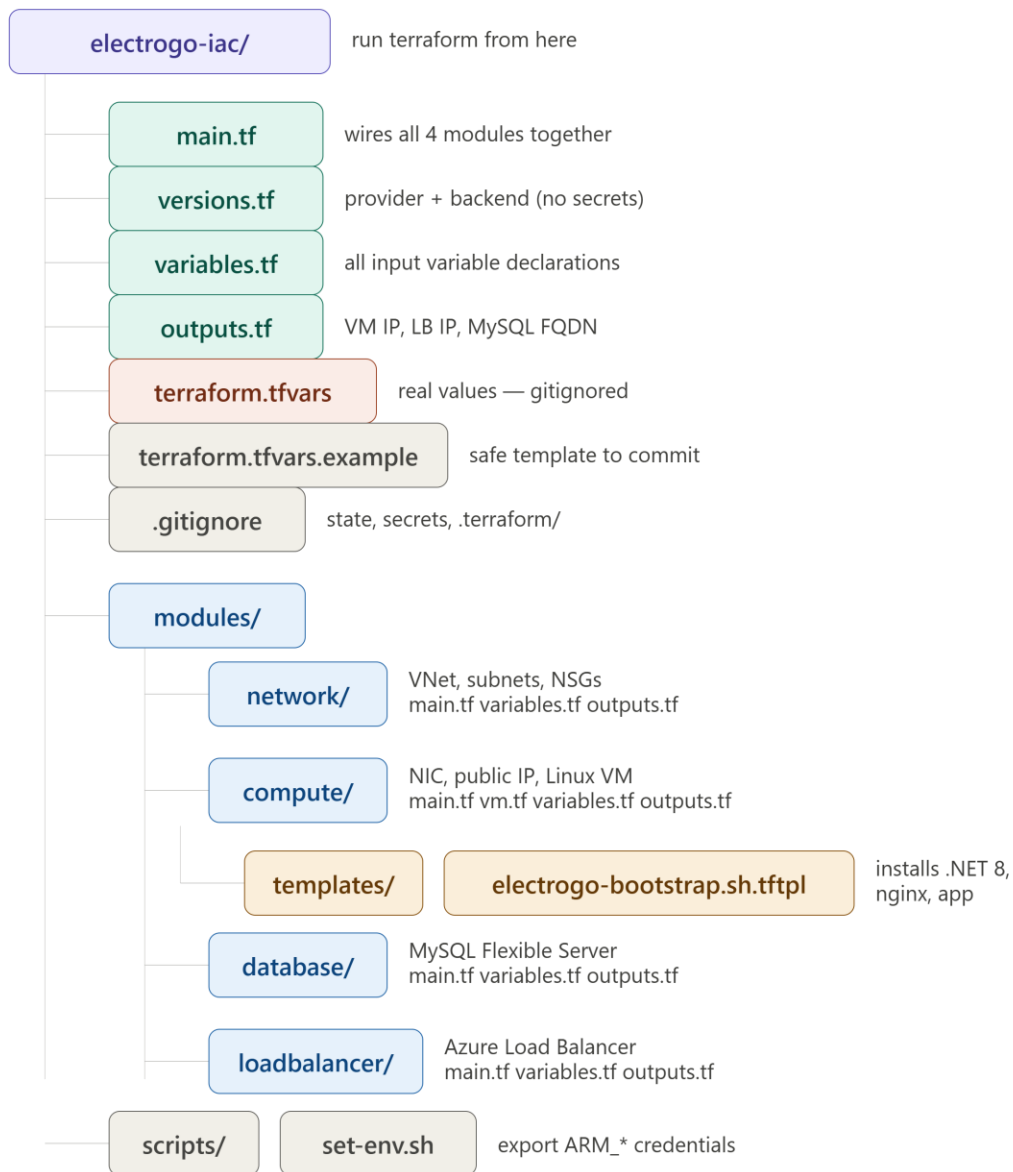
1. Open Azure DevOps.

2. Navigate to Repos.
3. Create the Electro GO repository.
4. Push the ASP.NET Core project.
5. Push Terraform files.



The screenshot shows the Azure DevOps web interface for the repository 'ElectroGo.git'. The left sidebar contains the 'Repos' section with 'Files' selected. The main area displays a list of files and folders with their last change dates and commit hashes.

Name	Last change	Commits
Data	16 Jun	7a2137a5 initial commit siva shankar
Migrations	20 Jun	bab064fa initial commit siva shankar
Models	16 Jun	7a2137a5 initial commit siva shankar
Pages	20 Jun	bab064fa initial commit siva shankar
Properties	16 Jun	7a2137a5 initial commit siva shankar
publish	20 Jun	bab064fa initial commit siva shankar
Terraform	3h ago	d06bde23 Updated main.tf kattasivashankar
wwwroot	16 Jun	7a2137a5 initial commit siva shankar
.gitignore	16 Jun	7a2137a5 initial commit siva shankar
appsettings.json	20 Jun	bab064fa initial commit siva shankar
ElectroGO.csproj	20 Jun	bab064fa initial commit siva shankar



7. Build Pipeline (Continuous Integration)

Purpose

The Build Pipeline compiles the ASP.NET Core application, restores project dependencies, executes tests, and publishes build artifacts for deployment.

Build Pipeline Stages

Stage 1 – Build

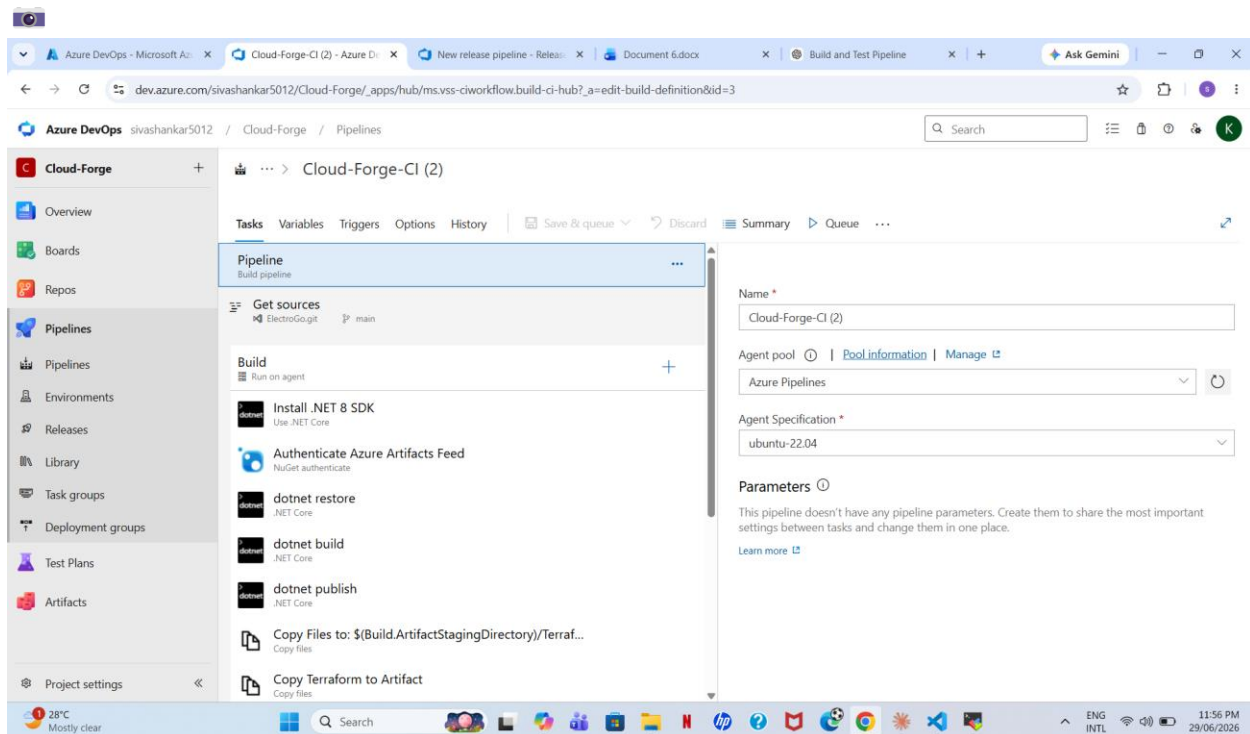
Aim

Compile the ASP.NET Core application, package the application files, and generate deployable artifacts for the release pipeline.

Tasks Performed

- Restore NuGet Packages (dotnet restore)
- Install .NET 8 SDK
- Authenticate Azure Artifacts Feed
- Build ASP.NET Core Project (dotnet build)
- Publish Application (dotnet publish)
- Copy Terraform Files to Artifact Staging Directory
- Package Terraform Files
- Push Package to Azure Artifacts (Universal Package)
- Publish Build Artifacts for Release Pipeline

Screenshot Placement



Build

 Run on agent



Install .NET 8 SDK

Use .NET Core



Authenticate Azure Artifacts Feed

NuGet authenticate



dotnet restore

.NET Core



dotnet build

.NET Core



dotnet publish

.NET Core



Copy Files to: \$(Build.ArtifactStagingDirectory)/Terraform...

Copy files



Copy Terraform to Artifact

Copy files



Push to Azure Artifacts (Universal Package)

Universal packages



Publish drop for Release

Publish Pipeline Artifacts

Test

 Run on agent



Authenticate Feed (Test Stage)

NuGet authenticate



Code Format Check

Bash



Verify Build

.NET Core

+

Jobs in run #46

▼	✓ Build	48s
✓	Initialize job	3s
✓	Pre-job: Push to Azur...	4s
✓	Checkout ElectroGo...	2s
✓	Install .NET 8 SDK	9s
✓	Authenticate Azure ...	<1s
✓	dotnet restore	8s
✓	dotnet build	7s
✓	dotnet publish	2s
✓	Copy Files to: /hom...	<1s
✓	Copy Terraform to ...	<1s
✓	Push to Azure Artific...	4s
✓	Publish drop for Rele...	2s
✓	Post-job: Checkout ...	<1s
✓	Finalize Job	<1s

✓ Build

```

1 Pool: Azure Pipelines
2 Image: ubuntu-22.04
3 Queued: Yesterday at 23:58 [manage_parallel_jobs]
4 Agent: Azure Pipelines 1
5 Started: Yesterday at 23:58
6 Duration: 48s
7
8 ► Condition evaluation
13
14 The agent request is already running or has already completed.
15 ► Job preparation parameters
37 ► fx 1 queue time variable used
40 📦 1 artifact produced
41 Job live console data:
42 Starting: Build
43 Async Command Start: DetectDockerContainer
44 Async Command End: DetectDockerContainer
45 Async Command Start: DetectDockerContainer
46 Async Command End: DetectDockerContainer
47 Finishing: Build

```

+

← **Jobs in run #46**
Cloud-Forge-CI (2)

Jobs

>	✓ Build	48s
∨	✓ Test	1m 1s
	✓ Initialize job	3s
	✓ Checkout ElectroGo.git...	2s
	✓ Authenticate Feed (Te...	<1s
	✓ Code Format Check	43s
	✓ Verify Build	10s
	✓ Post-job: Checkout El...	<1s
	✓ Finalize Job	<1s

✓ Build

```

1 Pool: Azure Pipelines
2 Image: ubuntu-22.04
3 Queued: Yesterday at 23:58 [manage_parallel_jobs]
4 Agent: Azure Pipelines 1
5 Started: Yesterday at 23:58
6 Duration: 48s
7
8 ▶ Condition evaluation
13
14 The agent request is already running or has already completed.
15 ▶ Job preparation parameters
37 ▶ fx 1 queue time variable used
40 1 artifact produced
41 Job live console data:
42 Starting: Build
43 Async Command Start: DetectDockerContainer
44 Async Command End: DetectDockerContainer
45 Async Command Start: DetectDockerContainer
46 Async Command End: DetectDockerContainer
47 Finishing: Build

```

Stage 2 – Test

Aim

Validate the build quality and ensure the application meets the required standards before deployment.

Tasks Performed

- Authenticate Azure Artifacts Feed (Test Stage)
- Execute Code Format Check
- Verify Build
- Confirm Successful Test Stage Execution

Screenshot

sivashankar5012 / Cloud-Forge / Pipelines / Cloud-Forge-CI (2) / 46

+

← Jobs in run #46
Cloud-Forge-CI (2)

Jobs

> ✓ Build 48s

✓ Test 1m 1s

✓ Initialize job 3s

✓ Checkout ElectroGo.git... 2s

✓ Authenticate Feed (Te... <1s

✓ Code Format Check 43s

✓ Verify Build 10s

✓ Post-job: Checkout El... <1s

✓ Finalize Job <1s

✓ Build

```
1 Pool: Azure Pipelines
2 Image: ubuntu-22.04
3 Queued: Yesterday at 23:58 [manage_parallel_jobs]
4 Agent: Azure Pipelines 1
5 Started: Yesterday at 23:58
6 Duration: 48s
7
8 ► Condition evaluation
13
14 The agent request is already running or has already completed.
15 ► Job preparation parameters
37 ► fx 1 queue time variable used
40 1 artifact produced
41 Job live console data:
42 Starting: Build
43 Async Command Start: DetectDockerContainer
44 Async Command End: DetectDockerContainer
45 Async Command Start: DetectDockerContainer
46 Async Command End: DetectDockerContainer
47 Finishing: Build
```



Code Format Check

[View raw log](#)

1 ▶ Condition evaluation

9 Starting: Code Format Check ⇄

10 =====

11 Task : Bash

12 Description : Run a Bash script on macOS, Linux, or Windows

13 Version : 3.274.1

14 Author : Microsoft Corporation

15 Help : <https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash>

16 =====

17 Generating script.

18 ===== Starting Command Output =====

19 /usr/bin/bash /home/vsts/work/_temp/a54c712d-86e6-498b-a90e-40ed2b1beeb2.sh

20 The dotnet runtime version is '10.0.9'.

21 Formatting code files in workspace '/home/vsts/work/1/s/ElectroGO.csproj'.

22 Loading workspace.

23 Determining projects to restore...

24 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore (>= 8.0.9) but Micr

25 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore.Design (>= 8.0.9) b

26 Restored /home/vsts/work/1/s/ElectroGO.csproj (in 12.44 sec).

27 Msbuild failed when processing the file '/home/vsts/work/1/s/ElectroGO.csproj' with message: ElectroGO depends on Microsoft.E

28 Msbuild failed when processing the file '/home/vsts/work/1/s/ElectroGO.csproj' with message: ElectroGO depends on Microsoft.E

29 Project ElectroGO is using configuration from '/home/vsts/work/1/s/obj/Debug/net8.0/ElectroGO.GeneratedMSBuildEditorConfig.ed

30 Project ElectroGO is using configuration from '/usr/share/dotnet/sdk/10.0.301/Sdks/Microsoft.NET.Sdk/analyzers/build/config/a

31 Complete in 32928ms.

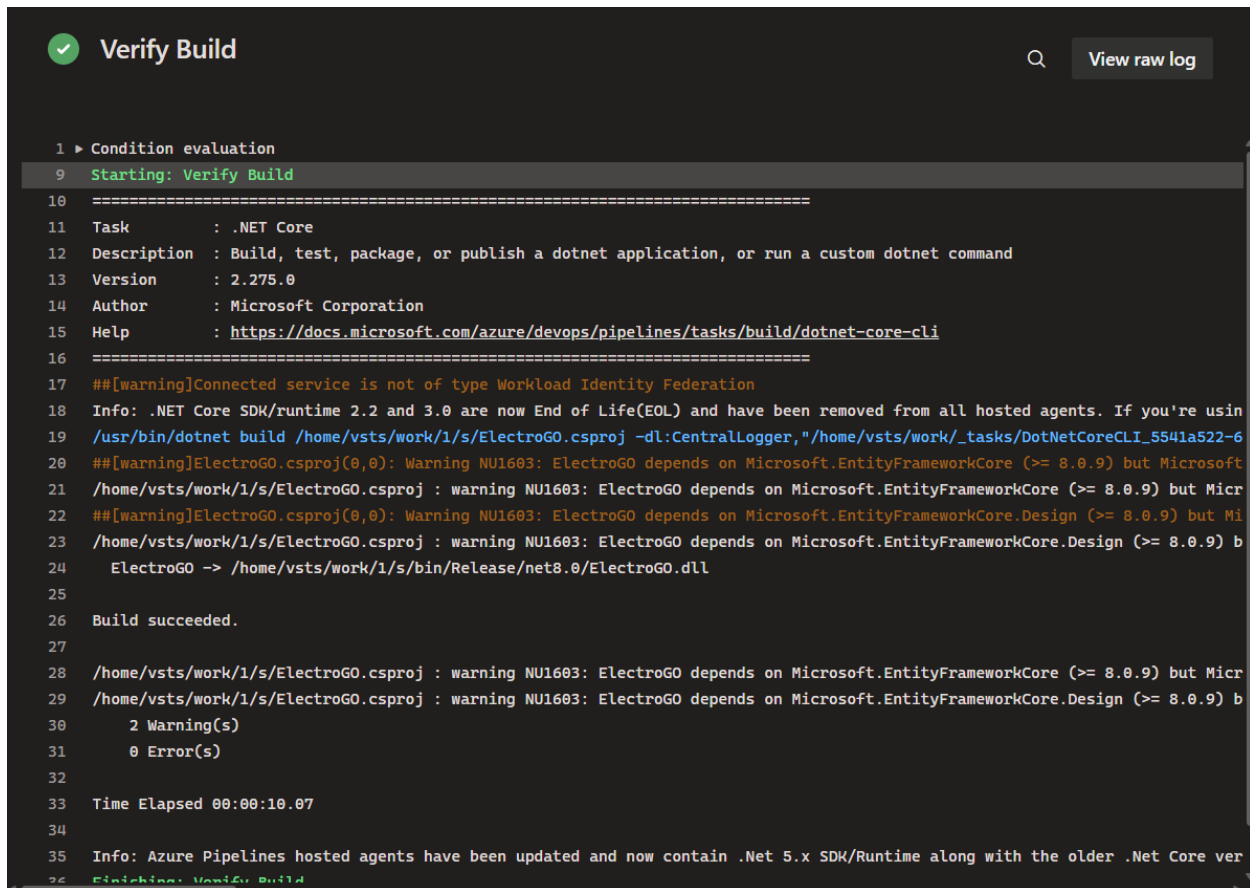
32 Determining formattable files.

33 Complete in 401ms.

34 Running formatters.

35 Running Code Style analysis.

36 Determining diagnostics



```
1 ▶ Condition evaluation
9 Starting: Verify Build
10 =====
11 Task      : .NET Core
12 Description : Build, test, package, or publish a dotnet application, or run a custom dotnet command
13 Version    : 2.275.0
14 Author     : Microsoft Corporation
15 Help       : https://docs.microsoft.com/azure/devops/pipelines/tasks/build/dotnet-core-cli
16 =====
17 ##[warning]Connected service is not of type Workload Identity Federation
18 Info: .NET Core SDK/runtime 2.2 and 3.0 are now End of Life(EOL) and have been removed from all hosted agents. If you're using
19 /usr/bin/dotnet build /home/vsts/work/1/s/ElectroGO.csproj -dl:CentralLogger,"/home/vsts/work/_tasks/DotNetCoreCLI_5541a522-6
20 ##[warning]ElectroGO.csproj(0,0): Warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore (>= 8.0.9) but Microsoft
21 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore (>= 8.0.9) but Micr
22 ##[warning]ElectroGO.csproj(0,0): Warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore.Design (>= 8.0.9) but Mi
23 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore.Design (>= 8.0.9) b
24 ElectroGO -> /home/vsts/work/1/s/bin/Release/net8.0/ElectroGO.dll
25
26 Build succeeded.
27
28 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore (>= 8.0.9) but Micr
29 /home/vsts/work/1/s/ElectroGO.csproj : warning NU1603: ElectroGO depends on Microsoft.EntityFrameworkCore.Design (>= 8.0.9) b
30 2 Warning(s)
31 0 Error(s)
32
33 Time Elapsed 00:00:10.07
34
35 Info: Azure Pipelines hosted agents have been updated and now contain .Net 5.x SDK/Runtime along with the older .Net Core ver
36 Finishing: Verify Build
```

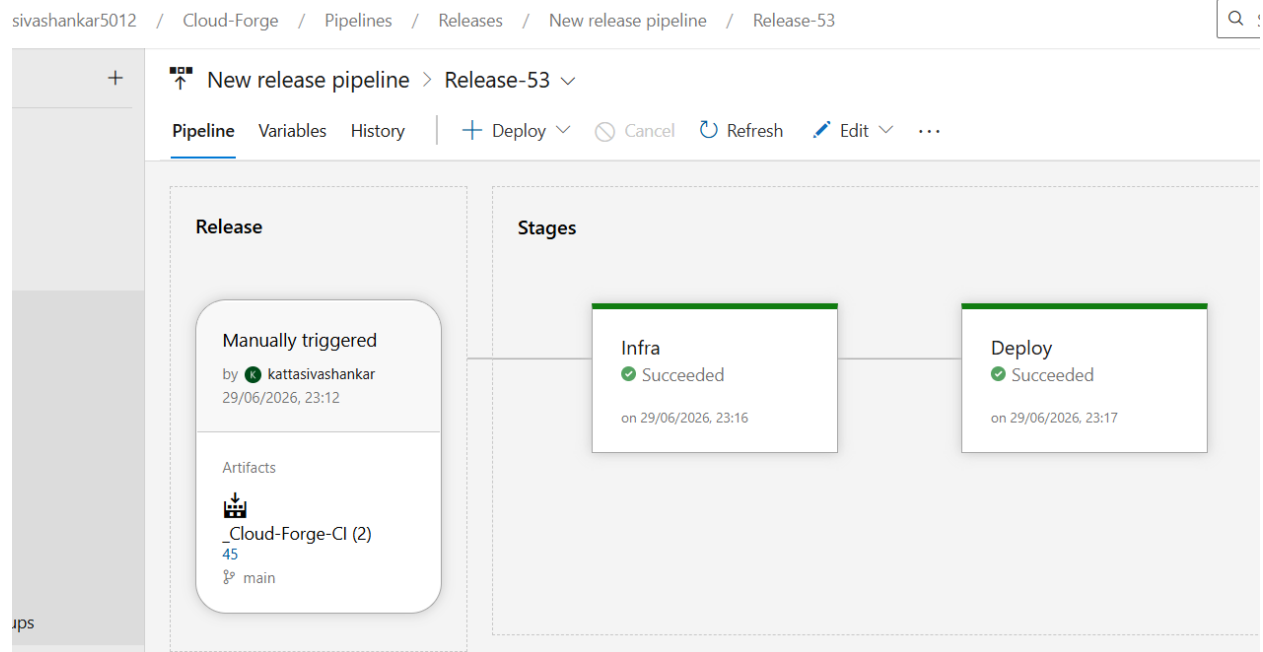
8. Release Pipeline (Continuous Deployment)

Purpose

The Release Pipeline automatically provisions Azure infrastructure using Terraform and deploys the Electro GO application.

Your screenshot already shows this stage.

Screenshot



Stage 1 – Terraform (Infrastructure Deployment)

Aim

Provision Azure infrastructure and configure the deployment environment using Terraform.

Tasks Performed

- Install Terraform 1.9.5
- Authenticate with Azure using Service Principal
- Execute Terraform commands to provision Azure infrastructure
- Capture Virtual Machine (VM) Public IP Address

Azure Resources Created

- Resource Group
- Virtual Network
- Public Subnet
- Private Subnet
- NSG
- Public IP
- Ubuntu VM

- MySQL Flexible Server
- Load Balancer
- Storage Account

Screenshot

Infra
Deployment process

Terraform Infra
Run on agent

Install Terraform 1.9.5
Bash

Capture VM Public IP
Bash

Bash ⓘ

Task version 3.*

Display name *

Install Terraform 1.9.5

Type ⓘ

☐ File Path ☒ Inline

Script *

```

set -e

# Install Terraform
wget -q https://releases.hashicorp.com/terraform/1.9.5/terraform_1.9.5_linux_amd64.zip
unzip -o -q terraform_1.9.5_linux_amd64.zip
sudo mv -f terraform /usr/local/bin/terraform
terraform version

# Login to Azure using Service Principal
az login \

```

Advanced ▾

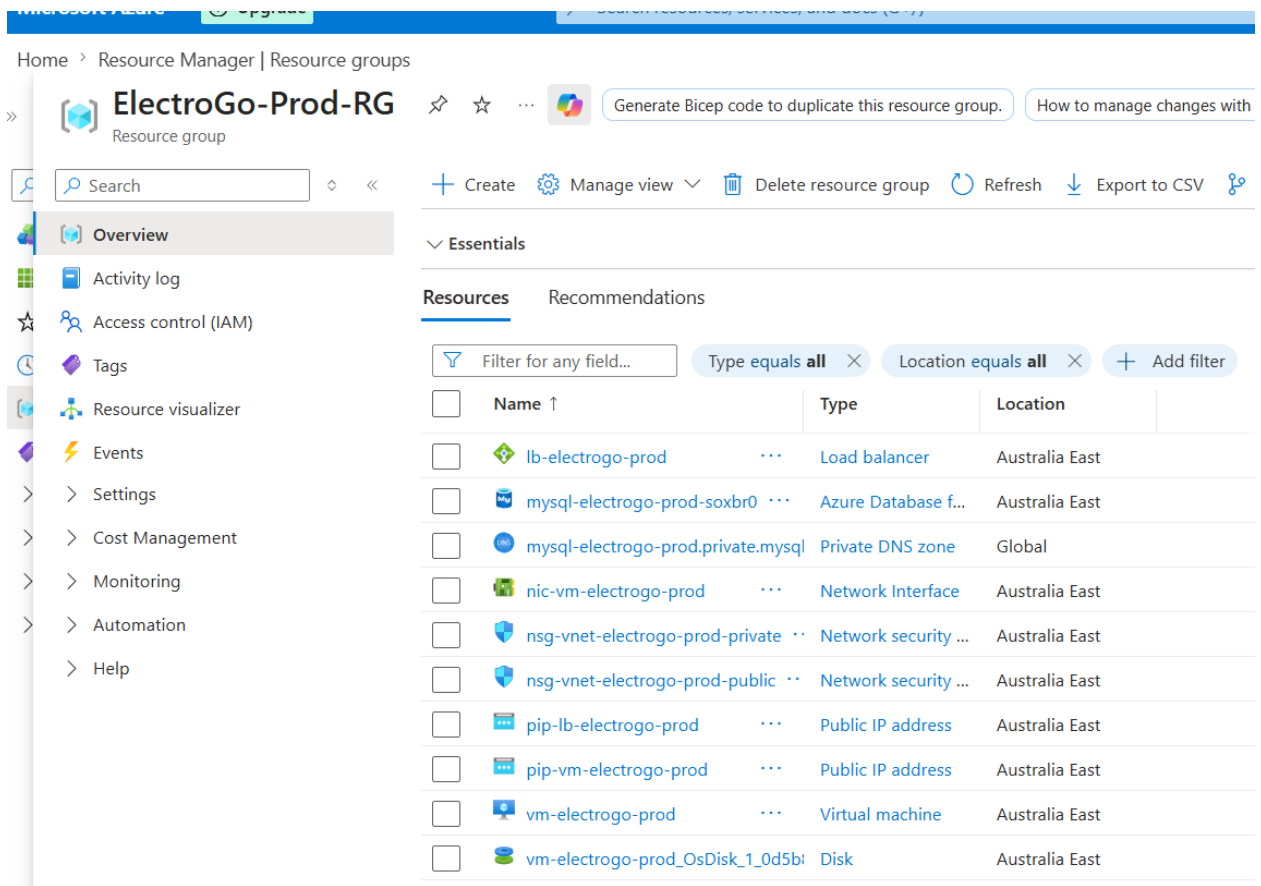
Deployment process
Succeeded

✓ **Terraform Infra**
Succeeded

Terraform Infra Started: 29/06/2026, 14:43:57

Pool: [Azure Pipelines](#) · Agent: Azure Pipelines 1 ... 10m 25s

✓ Initialize job · succeeded	4s
✓ Download artifact - _Cloud-Forge-CI (2) - electrogo-drop · succeeded	3s
✓ Install Terraform 1.9.5 · succeeded	10m 17s
✓ Capture VM Public IP · succeeded	<1s
✓ Finalize Job · succeeded	<1s



Stage 2 – Deploy

Aim

Deploy the ASP.NET Core application to the Ubuntu Virtual Machine using SSH.

Tasks Performed

- Download Build Artifacts
- Deploy Application to Ubuntu Virtual Machine via SSH
- Execute Deployment Commands on the VM
- Verify Successful Deployment

Screenshot Placement

+

All pipelines > New release pipeline

Pipeline

Tasks

Variables

Retention

Options

History

Deploy

Deployment process

Agent job

Run on agent

Download Build Artifacts

Download build artifacts

SSH Deploy to VM

Bash

Download build artifacts

Task version 1.*

Display name *
Download Build Artifacts

Download artifacts produced by *
☐ Current build ☒ Specific build

Project *
Cloud-Forge

Build pipeline *
Cloud-Forge-CI (2)

☐ When appropriate, download artifacts from the triggering build.

Build version to download *
Latest

☐ Download artifacts even from partial

Build Tags

View YAML

Remove

Power & Battery

Energy saver is on

+

New release pipeline > Release-54 > Deploy

Pipeline

Tasks

Variables

Logs

Tests

Deployment process

Succeeded

Agent job

Succeeded

Agent job

Pool: Azure Pipelines · Agent: Azure Pipelines 1

Started: 30/06/2026, 00:02:47

26s

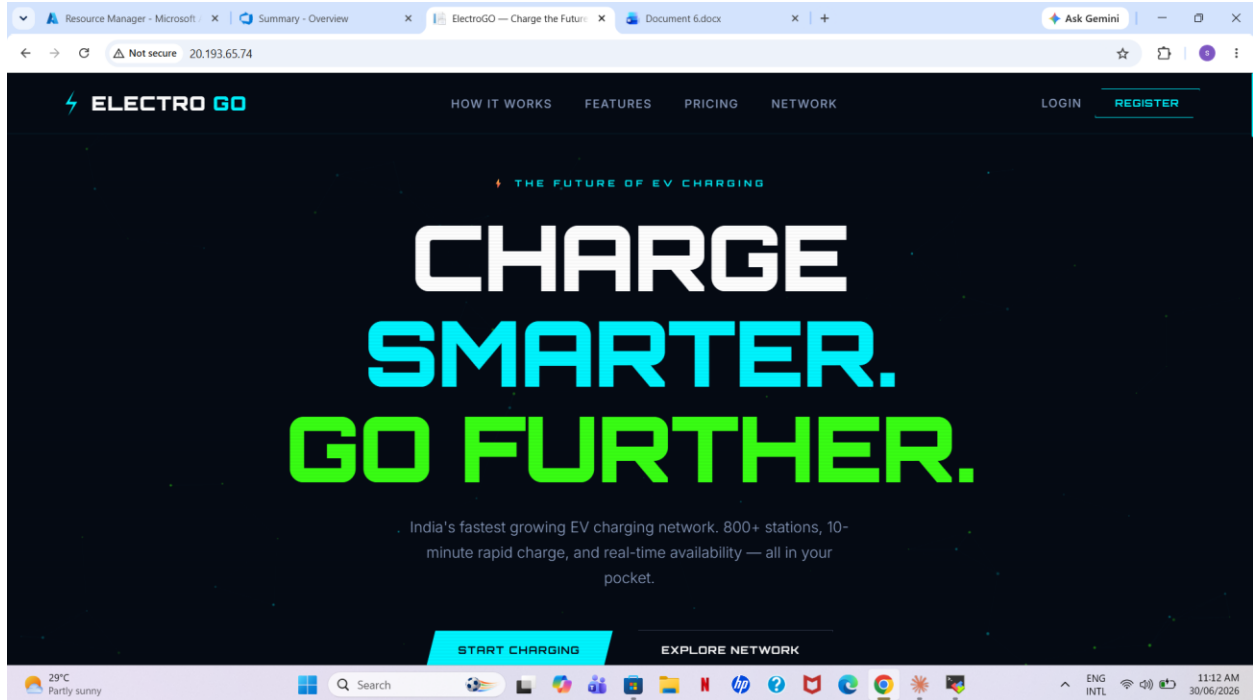
Initialize job · succeeded

Download artifact - _Cloud-Forge-CI (2) - electrogo-drop · succeeded

Download Build Artifacts · succeeded

SSH Deploy to VM · succeeded

Finalize Job · succeeded



9. Deployment Validation

Validation Checklist

Validation	Status
Source Code Available in Azure Repos	✓
Build Successful	✓
Tests Passed	✓
Terraform Successful	✓
Infrastructure Created	✓
Application Deployed	✓
Website Accessible	✓
Database Connected	✓

Screenshot

The screenshot displays the Azure portal interface for the resource group **ElectroGo-Prod-RG**. The top navigation bar includes the resource group name, a search bar, and links for [Export resource groups using Bicep or Terraform](#) and [How to manage changes with deployment to](#).

The left sidebar shows the **Overview** tab selected, with a list of navigation options: Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help.

The main content area is divided into two sections: **Essentials** and **Resources**. The **Resources** section is currently active, displaying a table of resources within the resource group.

Name	Type	Location
lb-electrogo-prod	Load balancer	Australia East
mysql-electrogo-prod-soxbr0	Azure Database f...	Australia East
mysql-electrogo-prod.private.mysql	Private DNS zone	Global
nic-vm-electrogo-prod	Network Interface	Australia East
nsg-vnet-electrogo-prod-private	Network security ...	Australia East
nsg-vnet-electrogo-prod-public	Network security ...	Australia East
pip-lb-electrogo-prod	Public IP address	Australia East
pip-vm-electrogo-prod	Public IP address	Australia East
vm-electrogo-prod	Virtual machine	Australia East

The **Details** section for the **vm-electrogo-prod** resource is shown below. It includes a list of actions (Connect, Start, Restart, Stop, Hibernate, Capture, Delete, Refresh, Scale, Open in mobile, Feedback, CLI / PS) and a **JSON View** link.

Essentials

Resource group (move): [ElectroGo-Prod-RG](#)

Status: Running

Location: Australia East

Subscription (move): [Azure subscription 1](#)

Subscription ID: 271f363b-85fa-4f55-bb1d-ceaaf4946f04

Operating system: Linux (ubuntu 24.04)

Size: Standard D2s v3 (2 vcpus, 8 GiB memory)

Primary NIC public IP: [20.193.65.74](#)

[1 associated public IPs](#)

Virtual network/subnet: [vnet-electrogo-prod/vnet-electrogo-public](#)

DNS name: [Not configured](#)

Health state: -

Time created: 29/06/2026, 17:45 UTC

Tags (edit): [environment : prod](#) [managed_by : terraform](#) [project : electrogo](#)

Properties

Virtual machine

Computer name: vm-electrogo-prod

Operating system: Linux (ubuntu 24.04)

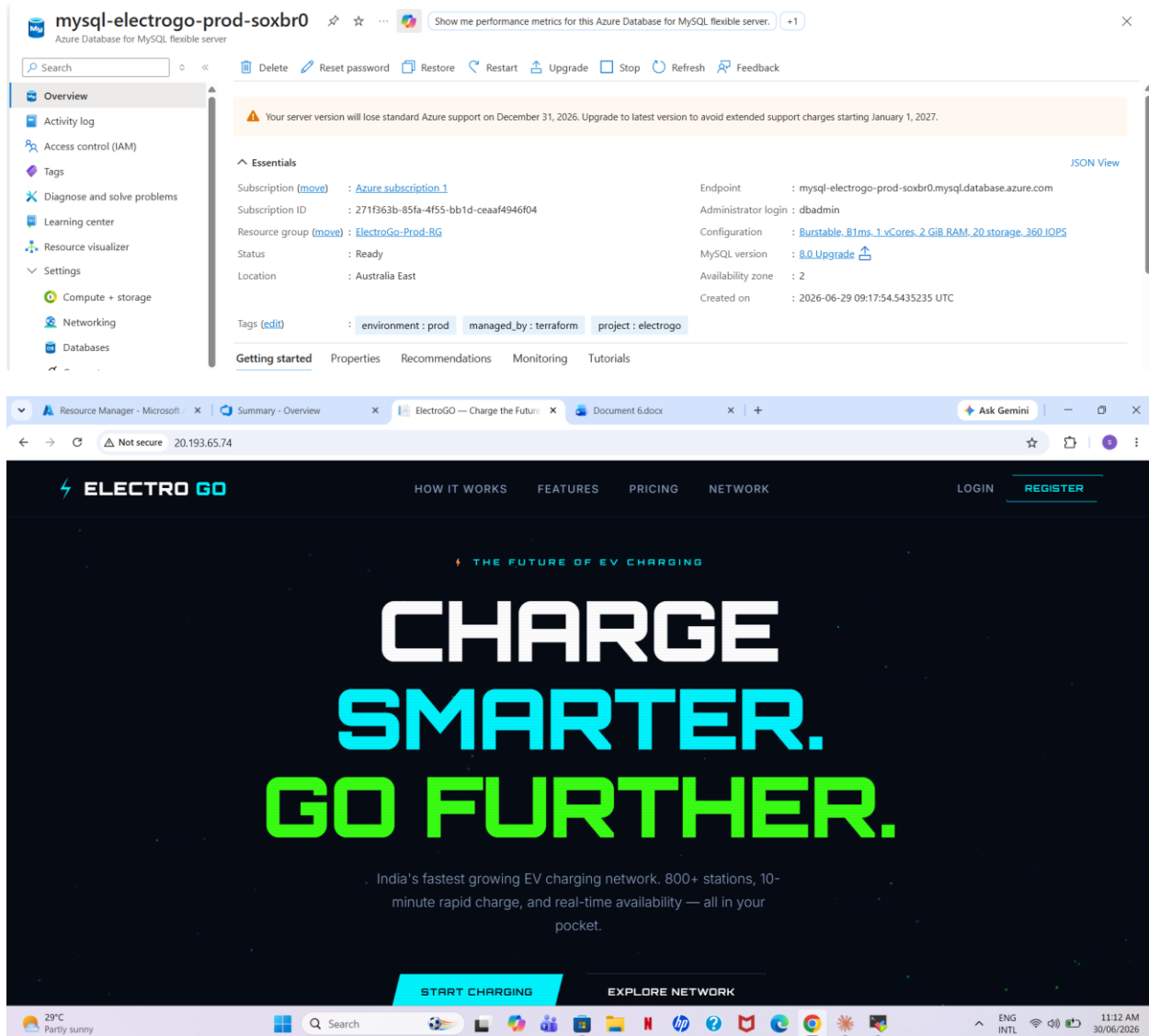
VM generation: V2

Networking

Public IP address: [20.193.65.74 \(Network interface nic-vm-electrogo-prod\)](#)

[1 associated public IPs](#)

Public IP address (IPv6): -



10. Conclusion

The Electro GO application was successfully deployed using Azure DevOps Classic CI/CD pipelines. Azure Repos served as the source code repository, while the Build Pipeline compiled and tested the application. The Release Pipeline automated infrastructure provisioning using Terraform and deployed the ASP.NET Core application to an Azure Virtual Machine. This implementation demonstrates a complete DevOps workflow, reducing manual effort, ensuring consistent deployments, and enabling Infrastructure as Code for efficient cloud resource management.